DBMS Project · Review II

A10 Hospital Management System

Database Implementation

Name	Pranav Praveen
Reg. No.	24BDE0137

Database	MySQL 8.4 · database `a10_hospital`
Objects	8 tables · 3 functions · 4 procedures (2 with cursors) · 4 triggers
Source files	`database/schema.sql`, `seed.sql`, `queries.sql`, `plsql.sql`
Date	28 October 2026

Evaluation criteria and where each is covered

#	Component	Criteria	What this project shows	Marks
1	DDL & Constraints	Tables with appropriate PK, FK and other constraints	8 tables, 12 foreign keys incl. a self-reference and a cascade, UNIQUE, CHECK, ENUM, DEFAULT	2
2	DML & Data Population	Meaningful insertion, modification and deletion	Linked sample data; INSERT / UPDATE / DELETE examples; constraints rejecting bad rows	1
3	SQL Queries	Joins, aggregation, subqueries, grouping	12 queries: inner, left and self joins; GROUP BY / HAVING; scalar, NOT IN, correlated subqueries	2
4	Procedures & Functions	Business operations as procedures and functions	Booking, prescription amendment (transaction), age, delivery fee, doctor earnings	2
5	Cursor & Triggers	Meaningful use in the application	Cursor-based order placement with stock check and rollback, cursor report; 4 business-rule triggers	2
6	Viva	Explain the SQL / PL-SQL and how it runs	Question-and-answer notes in section 6	1
	Total			**10**

1. DDL & Constraints (2 marks)

The database `a10_hospital` has exactly the **8 tables** of the ER diagram. Every table has a surrogate primary key (`INT AUTO_INCREMENT`), and every relationship in the ER diagram is a named `FOREIGN KEY`. Business rules are enforced inside the database with `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT` and `ENUM`, so invalid data is rejected no matter which program inserts it.

#	Table	Primary key	Foreign keys	Other constraints
1	`departments`	id	none	UNIQUE slug, UNIQUE name, DEFAULT is_active
2	`doctors`	id	department_id → departments	CHECK fee ≥ 0, experience ≥ 0, minutes > 0
3	`patients`	id	none	UNIQUE email, ENUM blood_group
4	`appointments`	id	patient_id, doctor_id, department_id	UNIQUE reference, CHECK ends_at > starts_at, ENUM status
5	`medicines`	id	none	UNIQUE slug, CHECK price ≥ 0, stock ≥ 0
6	`prescriptions`	id	patient_id, doctor_id, appointment_id, supersedes_id → prescriptions (self)	UNIQUE reference, CHECK version ≥ 1, ENUM status
7	`prescription_items`	id	prescription_id (**ON DELETE CASCADE**), medicine_id	CHECK quantity > 0 (weak entity)
8	`medication_orders`	id	patient_id, prescription_id	UNIQUE reference, CHECK total = subtotal + delivery_fee

Design points worth mentioning: `prescriptions.supersedes_id` is a **self-referencing foreign key** (an amended prescription points at the version it replaces); `prescription_items` is a **weak entity**, so its rows are deleted automatically with their prescription (`ON DELETE CASCADE`); `medicine_id` and `appointment_id` are nullable because those relationships are optional.

CREATE TABLE statements (database/schema.sql)

```sql
DROP DATABASE IF EXISTS a10_hospital;
CREATE DATABASE a10_hospital CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE a10_hospital;

-- 1. Department: a clinical department of the hospital
CREATE TABLE departments (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  slug        VARCHAR(60)  NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  services    VARCHAR(500),
  is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

-- 2. Doctor: belongs to one department
CREATE TABLE doctors (
  id                  INT AUTO_INCREMENT PRIMARY KEY,
  full_name           VARCHAR(100) NOT NULL,
  speciality          VARCHAR(100) NOT NULL,
  years_experience    INT NOT NULL DEFAULT 0,
  consultation_fee    DECIMAL(10,2) NOT NULL,
  consultation_minutes INT NOT NULL DEFAULT 15,
  department_id       INT NOT NULL,
  CONSTRAINT chk_doctor_experience CHECK (years_experience >= 0),
  CONSTRAINT chk_doctor_fee        CHECK (consultation_fee >= 0),
  CONSTRAINT chk_doctor_minutes    CHECK (consultation_minutes > 0),
  CONSTRAINT fk_doctor_department  FOREIGN KEY (department_id) REFERENCES departments(id)
);

-- 3. Patient: a registered user who receives care
CREATE TABLE patients (
  id                     INT AUTO_INCREMENT PRIMARY KEY,
  email                  VARCHAR(150) NOT NULL UNIQUE,
  full_name              VARCHAR(100) NOT NULL,
  phone                  VARCHAR(20),
  date_of_birth          DATE,
  blood_group            ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-'),
  address                VARCHAR(255),
  emergency_contact_name  VARCHAR(100),
  emergency_contact_phone VARCHAR(20)
);

-- 4. Appointment: a booked visit between one patient and one doctor
CREATE TABLE appointments (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  reference     VARCHAR(20) NOT NULL UNIQUE,
  starts_at     DATETIME NOT NULL,
  ends_at       DATETIME NOT NULL,
  status        ENUM('booked','completed','cancelled','no_show') NOT NULL DEFAULT 'booked',
  reason        VARCHAR(255),
  patient_id    INT NOT NULL,
  doctor_id     INT NOT NULL,
  department_id INT NOT NULL,
  CONSTRAINT chk_appointment_time      CHECK (ends_at > starts_at),
  CONSTRAINT fk_appointment_patient    FOREIGN KEY (patient_id)    REFERENCES patients(id),
  CONSTRAINT fk_appointment_doctor     FOREIGN KEY (doctor_id)     REFERENCES doctors(id),
  CONSTRAINT fk_appointment_department FOREIGN KEY (department_id) REFERENCES departments(id)
);

-- 5. Medicine: a product in the hospital pharmacy catalogue
CREATE TABLE medicines (
  id                   INT AUTO_INCREMENT PRIMARY KEY,
  slug                 VARCHAR(80)  NOT NULL UNIQUE,
  name                 VARCHAR(100) NOT NULL,
  category             VARCHAR(60)  NOT NULL,
  strength             VARCHAR(40),
```

```
  price                DECIMAL(10,2) NOT NULL,
  stock                INT NOT NULL DEFAULT 0,
  requires_prescription BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT chk_medicine_price CHECK (price >= 0),
  CONSTRAINT chk_medicine_stock CHECK (stock >= 0)
);

-- 6. Prescription: medicines a doctor issues to a patient.
--    It is amended by issuing a new version that points at the old one (supersedes_id).
CREATE TABLE prescriptions (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  reference      VARCHAR(20) NOT NULL UNIQUE,
  status         ENUM('active','superseded','cancelled') NOT NULL DEFAULT 'active',
  version        INT NOT NULL DEFAULT 1,
  issued_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  patient_id     INT NOT NULL,
  doctor_id      INT NOT NULL,
  appointment_id INT,
  supersedes_id  INT,
  CONSTRAINT chk_prescription_version   CHECK (version >= 1),
  CONSTRAINT fk_prescription_patient     FOREIGN KEY (patient_id)     REFERENCES patients(id),
  CONSTRAINT fk_prescription_doctor      FOREIGN KEY (doctor_id)      REFERENCES doctors(id),
  CONSTRAINT fk_prescription_appointment FOREIGN KEY (appointment_id) REFERENCES appointments(id),
  CONSTRAINT fk_prescription_supersedes  FOREIGN KEY (supersedes_id)  REFERENCES prescriptions(id)
);

-- 7. Prescription item: one medicine line on a prescription (weak entity: deleted with its prescription)
CREATE TABLE prescription_items (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  medication_name VARCHAR(100) NOT NULL,
  dosage          VARCHAR(60)  NOT NULL,
  frequency       VARCHAR(60)  NOT NULL,
  duration        VARCHAR(60)  NOT NULL,
  quantity        INT NOT NULL,
  prescription_id INT NOT NULL,
  medicine_id     INT,
  CONSTRAINT chk_item_quantity CHECK (quantity > 0),
  CONSTRAINT fk_item_prescription FOREIGN KEY (prescription_id) REFERENCES prescriptions(id) ON DELETE
CASCADE,
  CONSTRAINT fk_item_medicine     FOREIGN KEY (medicine_id)     REFERENCES medicines(id)
);

-- 8. Medication order: a patient's pharmacy order, optionally backed by a prescription
CREATE TABLE medication_orders (
  id               INT AUTO_INCREMENT PRIMARY KEY,
  reference        VARCHAR(20) NOT NULL UNIQUE,
  status           ENUM('placed','packed','dispatched','delivered','cancelled') NOT NULL DEFAULT 'placed',
  subtotal         DECIMAL(10,2) NOT NULL,
  delivery_fee     DECIMAL(10,2) NOT NULL DEFAULT 0,
  total            DECIMAL(10,2) NOT NULL,
  delivery_address VARCHAR(255) NOT NULL,
  patient_id       INT NOT NULL,
  prescription_id  INT,
  CONSTRAINT chk_order_amounts CHECK (subtotal >= 0 AND delivery_fee >= 0),
  CONSTRAINT chk_order_total   CHECK (total = subtotal + delivery_fee),
  CONSTRAINT fk_order_patient      FOREIGN KEY (patient_id)      REFERENCES patients(id),
  CONSTRAINT fk_order_prescription FOREIGN KEY (prescription_id) REFERENCES prescriptions(id)
);
```

Checking the structure

List the tables

```
SHOW TABLES;
```

OUTPUT

Tables_in_a10_hospital
appointments
departments
doctors
medication_orders

Tables_in_a10_hospital
medicines
patients
prescription_items
prescriptions

Structure of one table

Shows the columns, types, keys (PRI / UNI / MUL = foreign key index) and defaults.

```
DESCRIBE appointments;
```

OUTPUT

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	*NULL*	auto_increment
reference	varchar(20)	NO	UNI	*NULL*	
starts_at	datetime	NO		*NULL*	
ends_at	datetime	NO		*NULL*	
status	enum('booked','completed','cancelled','no_show')	NO		booked	
reason	varchar(255)	YES		*NULL*	
patient_id	int	NO	MUL	*NULL*	
doctor_id	int	NO	MUL	*NULL*	
department_id	int	NO	MUL	*NULL*	

All constraints in the database

Read from the data dictionary (`information_schema`).

```
SELECT TABLE_NAME, CONSTRAINT_TYPE, COUNT(*) AS how_many
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'a10_hospital'
GROUP BY TABLE_NAME, CONSTRAINT_TYPE
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
```

OUTPUT

TABLE_NAME	CONSTRAINT_TYPE	how_many
appointments	CHECK	1
appointments	FOREIGN KEY	3
appointments	PRIMARY KEY	1
appointments	UNIQUE	1
departments	PRIMARY KEY	1
departments	UNIQUE	2
doctors	CHECK	3
doctors	FOREIGN KEY	1
doctors	PRIMARY KEY	1
medication_orders	CHECK	2
medication_orders	FOREIGN KEY	2
medication_orders	PRIMARY KEY	1
medication_orders	UNIQUE	1
medicines	CHECK	2
medicines	PRIMARY KEY	1
medicines	UNIQUE	1
patients	PRIMARY KEY	1
patients	UNIQUE	1
prescription_items	CHECK	1
prescription_items	FOREIGN KEY	2

TABLE_NAME	CONSTRAINT_TYPE	how_many
prescription_items	PRIMARY KEY	1
prescriptions	CHECK	1
prescriptions	FOREIGN KEY	4
prescriptions	PRIMARY KEY	1
prescriptions	UNIQUE	1

2. DML & Data Population (1 mark)

The tables are filled with realistic sample data (`database/seed.sql`): six departments, eight doctors, six patients, appointments in every status, a prescription that has been amended to version 2, and pharmacy orders whose totals match their prescriptions. Rows are inserted parent-first (departments → doctors → appointments → prescriptions → items → orders) so every foreign key already exists.

Rows in each table after loading the sample data

```
SELECT 'departments' AS table_name, COUNT(*) AS rows_ FROM departments
UNION ALL SELECT 'doctors', COUNT(*) FROM doctors
UNION ALL SELECT 'patients', COUNT(*) FROM patients
UNION ALL SELECT 'appointments', COUNT(*) FROM appointments
UNION ALL SELECT 'medicines', COUNT(*) FROM medicines
UNION ALL SELECT 'prescriptions', COUNT(*) FROM prescriptions
UNION ALL SELECT 'prescription_items', COUNT(*) FROM prescription_items
UNION ALL SELECT 'medication_orders', COUNT(*) FROM medication_orders;
```

OUTPUT

table_name	rows_
departments	6
doctors	8
patients	6
appointments	8
medicines	8
prescriptions	4
prescription_items	8
medication_orders	4

Insertion

Register a new patient and book their first appointment

`LAST_INSERT_ID()` returns the id MySQL just generated for the patient, so the appointment can refer to it.

```
INSERT INTO patients (email, full_name, phone, date_of_birth, blood_group, address)
VALUES ('sanjay.v@example.com', 'Sanjay Verma', '9876500007', '1995-08-01', 'B-', '5 Civil Lines, Jaipur');
INSERT INTO appointments (reference, starts_at, ends_at, reason, patient_id, doctor_id, department_id)
VALUES ('APT-1009', '2026-11-05 11:00:00', '2026-11-05 11:10:00', 'Headache', LAST_INSERT_ID(), 6, 5);
SELECT a.reference, p.full_name AS patient, d.full_name AS doctor, a.starts_at, a.status
FROM appointments a JOIN patients p ON p.id = a.patient_id JOIN doctors d ON d.id = a.doctor_id
WHERE a.reference = 'APT-1009';
```

OUTPUT

```
Query OK, 1 row affected
Query OK, 1 row affected
```

reference	patient	doctor	starts_at	status
APT-1009	Sanjay Verma	Dr. Rohan Das	2026-11-05 11:00:00	booked

Modification

Mark a visit as completed and take medicine out of stock

```
UPDATE appointments SET status = 'completed' WHERE reference = 'APT-1007';
UPDATE medicines SET stock = stock - 10 WHERE slug = 'paracetamol-500';
SELECT name, stock FROM medicines WHERE slug = 'paracetamol-500';
```

OUTPUT

```
Query OK, 1 row affected
Query OK, 1 row affected
```

name	stock
Paracetamol	490

Raise the fee of every doctor in Cardiology by 10%

An UPDATE with a subquery in its WHERE clause.

```
UPDATE doctors SET consultation_fee = consultation_fee * 1.10
WHERE department_id = (SELECT id FROM departments WHERE slug = 'cardiology');
SELECT full_name, consultation_fee FROM doctors WHERE department_id = 1;
```

OUTPUT

```
Query OK, 2 rows affected
```

full_name	consultation_fee
Dr. Ananya Rao	990.00
Dr. Vikram Menon	770.00

Deletion

Cancel the appointment booked above by deleting it

```
DELETE FROM appointments WHERE reference = 'APT-1009';
SELECT COUNT(*) AS remaining FROM appointments WHERE reference = 'APT-1009';
```

OUTPUT

```
Query OK, 1 row affected
```

remaining
0

Cascading delete of a weak entity

Deleting prescription RX-2002 also deletes its medicine lines, because of `ON DELETE CASCADE`. Its pharmacy order has to go first, since that foreign key has no cascade. This runs inside a transaction and is undone with ROLLBACK so the data stays intact.

```
START TRANSACTION;
DELETE FROM medication_orders WHERE prescription_id = 2;
DELETE FROM prescriptions WHERE reference = 'RX-2002';
SELECT COUNT(*) AS items_left_for_rx_2002 FROM prescription_items WHERE prescription_id = 2;
ROLLBACK;
```

OUTPUT

```
Query OK, 0 rows affected

Query OK, 1 row affected

Query OK, 1 row affected
```

items_left_for_rx_2002
0

```
Query OK, 0 rows affected
```

Constraints rejecting bad data

Foreign key: department 99 does not exist

```
INSERT INTO doctors (full_name, speciality, consultation_fee, department_id) VALUES ('Dr. Test', 'Test',
100, 99);
```

OUTPUT

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`a10_hospital`.`doctors`, CONSTRAINT `fk_doctor_department` FOREIGN KEY (`department_id`) REFERENCES
`departments` (`id`))
```

UNIQUE: email already registered

```
INSERT INTO patients (email, full_name) VALUES ('rahul.sharma@example.com', 'Duplicate');
```

OUTPUT

```
ERROR 1062 (23000): Duplicate entry 'rahul.sharma@example.com' for key 'patients.email'
```

CHECK: price cannot be negative

```
INSERT INTO medicines (slug, name, category, price) VALUES ('bad', 'Bad', 'Test', -5);
```

OUTPUT

```
ERROR 3819 (HY000): Check constraint 'chk_medicine_price' is violated.
```

CHECK: an appointment must end after it starts

```
UPDATE appointments SET ends_at = starts_at WHERE reference = 'APT-1008';
```

OUTPUT

```
ERROR 3819 (HY000): Check constraint 'chk_appointment_time' is violated.
```

Sample data (database/seed.sql)

```
INSERT INTO departments (slug, name, description, services, is_active) VALUES
('cardiology',    'Cardiology',    'Heart and blood vessel care',          'ECG, Echocardiography, Stress test',
TRUE),
('orthopaedics',  'Orthopaedics',  'Bones, joints and sports injuries',    'Fracture care, Joint replacement, Physio',
TRUE),
('paediatrics',   'Paediatrics',   'Care for infants, children and teens', 'Vaccination, Growth checks',
TRUE),
('dermatology',   'Dermatology',   'Skin, hair and nail conditions',       'Acne care, Allergy testing',
TRUE),
('general-medicine', 'General Medicine', 'Everyday illness and check-ups', 'Health check-up, Fever clinic',         TRUE),
('neurology',     'Neurology',     'Brain, spine and nerve disorders',     'EEG, Headache clinic',
FALSE);


INSERT INTO doctors (full_name, speciality, years_experience, consultation_fee, consultation_minutes, department_id)
VALUES
('Dr. Ananya Rao',     'Interventional Cardiologist', 14, 900.00, 20, 1),
('Dr. Vikram Menon',   'Cardiologist',                 8, 700.00, 15, 1),
('Dr. Farhan Qureshi', 'Orthopaedic Surgeon',         12, 800.00, 20, 2),
('Dr. Meera Iyer',     'Paediatrician',               10, 600.00, 15, 3),
('Dr. Kavya Nair',     'Dermatologist',                6, 650.00, 15, 4),
('Dr. Rohan Das',      'General Physician',           18, 500.00, 10, 5),
('Dr. Sneha Kulkarni', 'General Physician',            4, 400.00, 10, 5),
('Dr. Arjun Pillai',   'Neurologist',                 15, 1000.00, 30, 6);


INSERT INTO patients (email, full_name, phone, date_of_birth, blood_group, address, emergency_contact_name,
emergency_contact_phone) VALUES
('rahul.sharma@example.com', 'Rahul Sharma', '9876500001', '1990-04-12', 'B+',  '12 MG Road, Bengaluru',      'Priya
Sharma', '9876500101'),
('aisha.khan@example.com',   'Aisha Khan',   '9876500002', '1985-09-23', 'O+',  '7 Park Street, Kolkata',     'Imran
Khan',   '9876500102'),
('karthik.r@example.com',    'Karthik R',    '9876500003', '2015-01-30', 'A+',  '45 Anna Nagar, Chennai',     'Lakshmi R',
'9876500103'),
('neha.gupta@example.com',   'Neha Gupta',   '9876500004', '1998-06-05', 'AB-', '3 Sector 15, Noida',         'Amit
Gupta',   '9876500104'),
('joseph.d@example.com',     'Joseph D''Souza', '9876500005', '1972-12-18', 'O-', '21 Panjim Road, Goa',      'Maria
D''Souza', '9876500105'),
('divya.patel@example.com',  'Divya Patel',  '9876500006', '2001-03-14', 'A-',  '9 CG Road, Ahmedabad',       'Ravi
Patel',   '9876500106');


INSERT INTO appointments (reference, starts_at, ends_at, status, reason, patient_id, doctor_id, department_id) VALUES
('APT-1001', '2026-10-20 10:00:00', '2026-10-20 10:20:00', 'completed', 'Chest pain on exertion',   1, 1, 1),
('APT-1002', '2026-10-21 11:00:00', '2026-10-21 11:20:00', 'completed', 'Knee pain after running',  4, 3, 2),
('APT-1003', '2026-10-22 09:30:00', '2026-10-22 09:45:00', 'completed', 'Routine vaccination',      3, 4, 3),
('APT-1004', '2026-10-23 16:00:00', '2026-10-23 16:10:00', 'completed', 'Fever and cough',          2, 6, 5),
('APT-1005', '2026-10-24 12:00:00', '2026-10-24 12:15:00', 'no_show',   'Skin rash',               6, 5, 4),
('APT-1006', '2026-10-26 15:00:00', '2026-10-26 15:10:00', 'cancelled', 'Annual check-up',          5, 7, 5),
('APT-1007', '2026-11-02 10:30:00', '2026-11-02 10:50:00', 'booked',    'Follow-up after ECG',     1, 1, 1),
('APT-1008', '2026-11-03 14:00:00', '2026-11-03 14:15:00', 'booked',    'Acne treatment',          6, 5, 4);


INSERT INTO medicines (slug, name, category, strength, price, stock, requires_prescription) VALUES
('paracetamol-500',  'Paracetamol',  'Pain relief',     '500 mg', 25.00, 500, FALSE),
('amoxicillin-500',  'Amoxicillin',  'Antibiotic',      '500 mg', 120.00, 200, TRUE),
('atorvastatin-10',  'Atorvastatin', 'Cardiac',         '10 mg',  95.00, 150, TRUE),
('aspirin-75',       'Aspirin',      'Cardiac',         '75 mg',  30.00, 300, FALSE),
('ibuprofen-400',    'Ibuprofen',    'Pain relief',     '400 mg', 40.00, 250, FALSE),
('cetirizine-10',    'Cetirizine',   'Allergy',         '10 mg',  20.00, 400, FALSE),
('azithromycin-500', 'Azithromycin', 'Antibiotic',      '500 mg', 110.00, 120, TRUE),
('vitamin-d3-60k',   'Vitamin D3',   'Supplement',      '60000 IU', 60.00, 180, FALSE);


-- Prescription 3 amends prescription 1 (version 2), so prescription 1 is marked superseded.
INSERT INTO prescriptions (reference, status, version, issued_at, patient_id, doctor_id, appointment_id, supersedes_id)
VALUES
('RX-2001', 'superseded', 1, '2026-10-20 10:25:00', 1, 1, 1, NULL),
('RX-2002', 'active',     1, '2026-10-21 11:25:00', 4, 3, 2, NULL),
('RX-2003', 'active',     2, '2026-10-21 09:00:00', 1, 1, 1, 1),
('RX-2004', 'active',     1, '2026-10-23 16:15:00', 2, 6, 4, NULL);


INSERT INTO prescription_items (medication_name, dosage, frequency, duration, quantity, prescription_id, medicine_id)
VALUES
('Atorvastatin', '1 tablet',  'Once at night',   '30 days', 30, 1, 3),
('Aspirin',      '1 tablet',  'Once daily',      '30 days', 30, 1, 4),
('Ibuprofen',    '1 tablet',  'Twice daily after food', '5 days', 10, 2, 5),
('Atorvastatin', '1 tablet',  'Once at night',   '60 days', 60, 3, 3),
('Aspirin',      '1 tablet',  'Once daily',      '60 days', 60, 3, 4),
('Paracetamol',  '1 tablet',  'Every 6 hours if fever', '3 days', 12, 4, 1),
('Azithromycin', '1 tablet',  'Once daily',      '3 days',  3, 4, 7),
('Steam inhalation', '10 minutes', 'Twice daily', '5 days', 10, 4, NULL);
```

```sql
INSERT INTO medication_orders (reference, status, subtotal, delivery_fee, total, delivery_address, patient_id,
prescription_id) VALUES
('ORD-3001', 'delivered',  7500.00, 0.00,  7500.00, '12 MG Road, Bengaluru',  1, 3),
('ORD-3002', 'dispatched',  400.00, 40.00,  440.00, '3 Sector 15, Noida',     4, 2),
('ORD-3003', 'placed',      630.00, 40.00,  670.00, '7 Park Street, Kolkata', 2, 4),
('ORD-3004', 'packed',      120.00, 40.00,  160.00, '9 CG Road, Ahmedabad',   6, NULL);
```

3. SQL Queries (2 marks)

Each query answers a question a hospital would actually ask. Together they cover inner, outer and self joins, aggregate functions, `GROUP BY` / `HAVING`, and simple, nested and correlated subqueries.

Doctors with their department

INNER JOIN · **ORDER BY**

Inner join between `doctors` and `departments` on the foreign key.

```sql
SELECT d.full_name, d.speciality, dep.name AS department, d.consultation_fee
FROM doctors d JOIN departments dep ON dep.id = d.department_id
ORDER BY dep.name, d.full_name;
```

OUTPUT

full_name	speciality	department	consultation_fee
Dr. Ananya Rao	Interventional Cardiologist	Cardiology	990.00
Dr. Vikram Menon	Cardiologist	Cardiology	770.00
Dr. Kavya Nair	Dermatologist	Dermatology	650.00
Dr. Rohan Das	General Physician	General Medicine	500.00
Dr. Sneha Kulkarni	General Physician	General Medicine	400.00
Dr. Arjun Pillai	Neurologist	Neurology	1000.00
Dr. Farhan Qureshi	Orthopaedic Surgeon	Orthopaedics	800.00
Dr. Meera Iyer	Paediatrician	Paediatrics	600.00

Upcoming appointments with patient and doctor names

multi-table JOIN · **WHERE**

Three-table join, filtered on status.

```sql
SELECT a.reference, a.starts_at, p.full_name AS patient, d.full_name AS doctor, a.reason
FROM appointments a
JOIN patients p ON p.id = a.patient_id
JOIN doctors d  ON d.id = a.doctor_id
WHERE a.status = 'booked'
ORDER BY a.starts_at;
```

OUTPUT

reference	starts_at	patient	doctor	reason
APT-1008	2026-11-03 14:00:00	Divya Patel	Dr. Kavya Nair	Acne treatment

Number of doctors per department, including empty ones

LEFT JOIN · **GROUP BY** · **COUNT**

A LEFT JOIN keeps departments that have no doctors; `COUNT(d.id)` counts them as 0.

```sql
SELECT dep.name AS department, COUNT(d.id) AS doctors
FROM departments dep LEFT JOIN doctors d ON d.department_id = dep.id
GROUP BY dep.id, dep.name
ORDER BY doctors DESC, dep.name;
```

OUTPUT

department	doctors
Cardiology	2
General Medicine	2
Dermatology	1
Neurology	1
Orthopaedics	1
Paediatrics	1

Appointments by status

GROUP BY · COUNT

```
SELECT status, COUNT(*) AS appointments
FROM appointments GROUP BY status ORDER BY appointments DESC;
```

OUTPUT

status	appointments
completed	5
no_show	1
cancelled	1
booked	1

Each prescription with its medicine lines

JOIN

```
SELECT rx.reference, rx.version, rx.status, i.medication_name, i.dosage, i.frequency, i.quantity
FROM prescriptions rx JOIN prescription_items i ON i.prescription_id = rx.id
ORDER BY rx.reference, i.id;
```

OUTPUT

reference	version	status	medication_name	dosage	frequency	quantity
RX-2001	1	superseded	Atorvastatin	1 tablet	Once at night	30
RX-2001	1	superseded	Aspirin	1 tablet	Once daily	30
RX-2002	1	active	Ibuprofen	1 tablet	Twice daily after food	10
RX-2003	2	active	Atorvastatin	1 tablet	Once at night	60
RX-2003	2	active	Aspirin	1 tablet	Once daily	60
RX-2004	1	active	Paracetamol	1 tablet	Every 6 hours if fever	12
RX-2004	1	active	Azithromycin	1 tablet	Once daily	3
RX-2004	1	active	Steam inhalation	10 minutes	Twice daily	10

Which version each amended prescription replaced

SELF JOIN

A self join: the `prescriptions` table is joined to itself through `supersedes_id`.

```
SELECT new_rx.reference AS new_version, new_rx.version, old_rx.reference AS replaces, old_rx.status AS
old_status
FROM prescriptions new_rx JOIN prescriptions old_rx ON old_rx.id = new_rx.supersedes_id;
```

OUTPUT

new_version	version	replaces	old_status
RX-2003	2	RX-2001	superseded

Patients who spent more than Rs 200 at the pharmacy

JOIN · SUM · GROUP BY · HAVING

`WHERE` filters rows before grouping; `HAVING` filters the groups after `SUM`.

```
SELECT p.full_name, COUNT(o.id) AS orders, SUM(o.total) AS total_spent
FROM patients p JOIN medication_orders o ON o.patient_id = p.id
WHERE o.status <> 'cancelled'
GROUP BY p.id, p.full_name
HAVING SUM(o.total) > 200
ORDER BY total_spent DESC;
```

OUTPUT

full_name	orders	total_spent
Rahul Sharma	1	7500.00
Aisha Khan	1	670.00
Neha Gupta	1	440.00

Pharmacy catalogue summary by category

MIN / MAX / AVG / SUM · GROUP BY

```
SELECT category, COUNT(*) AS medicines, MIN(price) AS cheapest, MAX(price) AS dearest,
       ROUND(AVG(price), 2) AS avg_price, SUM(stock) AS units_in_stock
FROM medicines GROUP BY category ORDER BY category;
```

OUTPUT

category	medicines	cheapest	dearest	avg_price	units_in_stock
Allergy	1	20.00	20.00	20.00	400
Antibiotic	2	110.00	120.00	115.00	320
Cardiac	2	30.00	95.00	62.50	450
Pain relief	2	25.00	40.00	32.50	740
Supplement	1	60.00	60.00	60.00	180

Medicines that have never been prescribed

subquery · NOT IN

Subquery with `NOT IN`.

```
SELECT name, strength FROM medicines
WHERE id NOT IN (SELECT medicine_id FROM prescription_items WHERE medicine_id IS NOT NULL);
```

OUTPUT

name	strength
Amoxicillin	500 mg
Cetirizine	10 mg
Vitamin D3	60000 IU

Doctors who charge more than the average fee

scalar subquery · AVG

The inner query computes one value (the average) that the outer query compares against.

```
SELECT full_name, consultation_fee FROM doctors
WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM doctors)
ORDER BY consultation_fee DESC;
```

OUTPUT

full_name	consultation_fee
Dr. Arjun Pillai	1000.00
Dr. Ananya Rao	990.00
Dr. Farhan Qureshi	800.00
Dr. Vikram Menon	770.00

Patients who have never placed a pharmacy order

correlated subquery · NOT EXISTS

A correlated subquery: it is re-evaluated for each patient `p`.

```
SELECT p.full_name, p.email FROM patients p
WHERE NOT EXISTS (SELECT 1 FROM medication_orders o WHERE o.patient_id = p.id);
```

OUTPUT

full_name	email
Karthik R	karthik.r@example.com
Joseph D'Souza	joseph.d@example.com
Sanjay Verma	sanjay.v@example.com

Busiest doctor in each department

GROUP BY · HAVING · correlated subquery · ALL

A correlated subquery with `>= ALL` picks the doctor(s) with the most appointments in their own department.

```
SELECT dep.name AS department, d.full_name, COUNT(a.id) AS appointments
FROM doctors d
JOIN departments dep ON dep.id = d.department_id
LEFT JOIN appointments a ON a.doctor_id = d.id
GROUP BY dep.name, d.id, d.full_name, d.department_id
HAVING COUNT(a.id) >= ALL (SELECT COUNT(a2.id) FROM doctors d2
                           LEFT JOIN appointments a2 ON a2.doctor_id = d2.id
                           WHERE d2.department_id = d.department_id
                           GROUP BY d2.id)
ORDER BY dep.name;
```

OUTPUT

department	full_name	appointments
Cardiology	Dr. Ananya Rao	2
Dermatology	Dr. Kavya Nair	2
General Medicine	Dr. Rohan Das	1
General Medicine	Dr. Sneha Kulkarni	1
Neurology	Dr. Arjun Pillai	0
Orthopaedics	Dr. Farhan Qureshi	1
Paediatrics	Dr. Meera Iyer	1

4. Procedures & Functions (2 marks)

All of these are in `database/plsql.sql`. They work on the existing 8 tables (no extra tables). A **function** returns one value and can be used inside a `SELECT`. A **procedure** carries out a whole business operation, returns results through `OUT` parameters, and stops the operation with `SIGNAL` when a rule is broken.

Object	Business operation
`fn_patient_age(dob)`	Age in whole years, for listing patients
`fn_delivery_fee(subtotal)`	Pharmacy delivery charge: free above Rs 1000, otherwise Rs 40
`fn_doctor_earnings(doctor_id)`	Consultation fees a doctor earned from completed appointments
`sp_book_appointment(...)`	Books a visit: works out the end time and department, rejects past times and clashes
`sp_amend_prescription(...)`	Issues version *n*+1 of a prescription, copies its lines and supersedes the old one, as one transaction

Functions

```
-- Age in whole years from a date of birth (NULL if the date is unknown).
CREATE FUNCTION fn_patient_age(p_dob DATE)
RETURNS INT
NOT DETERMINISTIC NO SQL
BEGIN
  IF p_dob IS NULL THEN
    RETURN NULL;
  END IF;
  RETURN TIMESTAMPDIFF(YEAR, p_dob, CURDATE());
END$$

-- Pharmacy delivery charge: free for orders above Rs 1000, otherwise a flat Rs 40.
CREATE FUNCTION fn_delivery_fee(p_subtotal DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC NO SQL
BEGIN
  IF p_subtotal > 1000 THEN
    RETURN 0.00;
  END IF;
  RETURN 40.00;
END$$

-- Consultation fees a doctor has earned from completed appointments.
CREATE FUNCTION fn_doctor_earnings(p_doctor_id INT)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
  DECLARE v_total DECIMAL(10,2);
  SELECT COALESCE(SUM(d.consultation_fee), 0) INTO v_total
  FROM appointments a JOIN doctors d ON d.id = a.doctor_id
  WHERE a.doctor_id = p_doctor_id AND a.status = 'completed';
  RETURN v_total;
END$$
```

Using the functions in queries

```
SELECT full_name, date_of_birth, fn_patient_age(date_of_birth) AS age FROM patients ORDER BY age DESC;
SELECT full_name, fn_doctor_earnings(id) AS earnings FROM doctors ORDER BY earnings DESC, full_name;
SELECT fn_delivery_fee(400) AS fee_for_400, fn_delivery_fee(7500) AS fee_for_7500;
```

OUTPUT

full_name	date_of_birth	age
Joseph D'Souza	1972-12-18	53
Aisha Khan	1985-09-23	41
Rahul Sharma	1990-04-12	36
Sanjay Verma	1995-08-01	31
Neha Gupta	1998-06-05	28

full_name	date_of_birth	age
Divya Patel	2001-03-14	25
Karthik R	2015-01-30	11

full_name	earnings
Dr. Ananya Rao	1980.00
Dr. Farhan Qureshi	800.00
Dr. Meera Iyer	600.00
Dr. Rohan Das	500.00
Dr. Arjun Pillai	0.00
Dr. Kavya Nair	0.00
Dr. Sneha Kulkarni	0.00
Dr. Vikram Menon	0.00

fee_for_400	fee_for_7500
40.00	0.00

Procedure: book an appointment

```
-- Books an appointment. The end time comes from the doctor's consultation length and the
-- department from the doctor. Rejects past times and slots that clash with the doctor's
-- other booked appointments. Returns the new reference (e.g. APT-1009) in p_reference.
CREATE PROCEDURE sp_book_appointment(
  IN  p_patient_id INT,
  IN  p_doctor_id  INT,
  IN  p_starts_at  DATETIME,
  IN  p_reason     VARCHAR(255),
  OUT p_reference  VARCHAR(20))
BEGIN
  DECLARE v_minutes INT;
  DECLARE v_department_id INT;
  DECLARE v_ends_at DATETIME;

  SELECT consultation_minutes, department_id INTO v_minutes, v_department_id
  FROM doctors WHERE id = p_doctor_id;
  IF v_minutes IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such doctor';
  END IF;
  IF p_starts_at <= NOW() THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Appointments must be booked in the future';
  END IF;

  SET v_ends_at = p_starts_at + INTERVAL v_minutes MINUTE;
  IF EXISTS (SELECT 1 FROM appointments
             WHERE doctor_id = p_doctor_id AND status = 'booked'
               AND starts_at < v_ends_at AND ends_at > p_starts_at) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The doctor already has an appointment at that time';
  END IF;

  SELECT CONCAT('APT-', COALESCE(MAX(CAST(SUBSTRING(reference, 5) AS UNSIGNED)), 1000) + 1)
  INTO p_reference FROM appointments;

  INSERT INTO appointments (reference, starts_at, ends_at, reason, patient_id, doctor_id, department_id)
  VALUES (p_reference, p_starts_at, v_ends_at, p_reason, p_patient_id, p_doctor_id, v_department_id);
END$$
```

Booking a slot

Only the start time is given. The procedure looks up the doctor's 10-minute consultation length and department, and generates the next reference.

```
CALL sp_book_appointment(2, 6, '2026-11-10 10:00:00', 'Follow-up for fever', @apt);
SELECT @apt AS new_reference;
SELECT reference, starts_at, ends_at, status, patient_id, doctor_id, department_id FROM appointments WHERE
reference = @apt;
```

OUTPUT

```
Query OK, 1 row affected
```

new_reference
APT-1009

reference	starts_at	ends_at	status	patient_id	doctor_id	department_id
APT-1009	2026-11-10 10:00:00	2026-11-10 10:10:00	booked	2	6	5

Rejected: the doctor is already busy at 10:05

```
CALL sp_book_appointment(3, 6, '2026-11-10 10:05:00', 'Cough', @x);
```

OUTPUT

```
ERROR 1644 (45000): The doctor already has an appointment at that time
```

Rejected: a time in the past

```
CALL sp_book_appointment(3, 6, '2026-01-01 10:00:00', 'Cough', @x);
```

OUTPUT

```
ERROR 1644 (45000): Appointments must be booked in the future
```

Procedure: amend a prescription (transaction)

```
-- Amends a prescription: issues a new version with a copy of its medicine lines and marks
-- the old one superseded, all in one transaction. Returns the new reference in p_new_reference.
CREATE PROCEDURE sp_amend_prescription(
  IN  p_reference     VARCHAR(20),
  OUT p_new_reference VARCHAR(20))
BEGIN
  DECLARE v_old_id INT;
  DECLARE v_new_id INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  SELECT id INTO v_old_id FROM prescriptions WHERE reference = p_reference FOR UPDATE;
  IF v_old_id IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such prescription';
  END IF;

  SELECT CONCAT('RX-', MAX(CAST(SUBSTRING(reference, 4) AS UNSIGNED)) + 1)
  INTO p_new_reference FROM prescriptions;

  -- trg_prescription_before_insert checks the old version is active and sets the version number.
  INSERT INTO prescriptions (reference, patient_id, doctor_id, appointment_id, supersedes_id)
  SELECT p_new_reference, patient_id, doctor_id, appointment_id, id
  FROM prescriptions WHERE id = v_old_id;
  SET v_new_id = LAST_INSERT_ID();

  INSERT INTO prescription_items (medication_name, dosage, frequency, duration, quantity, prescription_id,
medicine_id)
  SELECT medication_name, dosage, frequency, duration, quantity, v_new_id, medicine_id
  FROM prescription_items WHERE prescription_id = v_old_id;

  UPDATE prescriptions SET status = 'superseded' WHERE id = v_old_id;
  COMMIT;
END$$
```

Amending RX-2004

The new version gets the next reference, version 2 (set by the trigger in section 5), a copy of every line, and RX-2004 becomes superseded. If any step fails, the EXIT HANDLER rolls everything back.

```
CALL sp_amend_prescription('RX-2004', @rx);
SELECT @rx AS new_reference;
SELECT reference, status, version, supersedes_id FROM prescriptions ORDER BY id;
SELECT rx.reference, i.medication_name, i.quantity FROM prescription_items i JOIN prescriptions rx ON rx.id
= i.prescription_id WHERE rx.reference = @rx;
```

OUTPUT

```
Query OK, 0 rows affected
```

new_reference
RX-2005

reference	status	version	supersedes_id
RX-2001	superseded	1	*NULL*
RX-2002	active	1	*NULL*
RX-2003	active	2	1
RX-2004	superseded	1	*NULL*
RX-2005	active	2	4

reference	medication_name	quantity
RX-2005	Paracetamol	12

reference	medication_name	quantity
RX-2005	Azithromycin	3
RX-2005	Steam inhalation	10

5. Cursors & Triggers (2 marks)

A **cursor** lets a procedure handle a query's rows *one at a time*: `DECLARE` the cursor and a `NOT FOUND` handler, `OPEN` it, `FETCH` rows in a loop until the handler sets the done flag, then `CLOSE` it. A **trigger** runs automatically before or after an `INSERT`, `UPDATE` or `DELETE` on a table, and can read the row being changed through `NEW` and `OLD`.

Cursor 1: place a pharmacy order (sp_place_order)

The cursor walks through the medicine lines of a prescription. For each line it locks the medicine row (`FOR UPDATE`), checks there is enough stock, adds *price × quantity* to the subtotal and reduces the stock. If any medicine is short, the whole order is rolled back, including the stock already taken for earlier lines.

```
-- Places a pharmacy order for an active prescription. A CURSOR walks through the
-- prescription's medicine lines: for each one it checks the stock, adds price x quantity to
-- the subtotal and takes the quantity out of stock. If any medicine is short, the whole order
-- is rolled back. Lines with no catalogue medicine (e.g. steam inhalation) are skipped.
CREATE PROCEDURE sp_place_order(
  IN  p_prescription_ref VARCHAR(20),
  IN  p_address          VARCHAR(255),
  OUT p_order_reference  VARCHAR(20))
BEGIN
  DECLARE v_prescription_id INT;
  DECLARE v_patient_id INT;
  DECLARE v_status VARCHAR(20);
  DECLARE v_subtotal DECIMAL(10,2) DEFAULT 0;
  DECLARE v_medicine_id INT;
  DECLARE v_quantity INT;
  DECLARE v_price DECIMAL(10,2);
  DECLARE v_stock INT;
  DECLARE v_name VARCHAR(100);
  DECLARE v_done BOOLEAN DEFAULT FALSE;

  DECLARE item_cursor CURSOR FOR
    SELECT medicine_id, quantity FROM prescription_items
    WHERE prescription_id = v_prescription_id AND medicine_id IS NOT NULL;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  SELECT id, patient_id, status INTO v_prescription_id, v_patient_id, v_status
  FROM prescriptions WHERE reference = p_prescription_ref;
  IF v_prescription_id IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No such prescription';
  END IF;
  IF v_status <> 'active' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only an active prescription can be ordered';
  END IF;

  OPEN item_cursor;
  read_loop: LOOP
    FETCH item_cursor INTO v_medicine_id, v_quantity;
    IF v_done THEN
      LEAVE read_loop;
    END IF;

    SELECT name, price, stock INTO v_name, v_price, v_stock
    FROM medicines WHERE id = v_medicine_id FOR UPDATE;
    IF v_stock < v_quantity THEN
      SET @msg = CONCAT('Not enough stock of ', v_name, ' (', v_stock, ' left, ', v_quantity, ' needed)');
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = @msg;
    END IF;

    SET v_subtotal = v_subtotal + v_price * v_quantity;
    UPDATE medicines SET stock = stock - v_quantity WHERE id = v_medicine_id;
```

```
  END LOOP;
  CLOSE item_cursor;

  SELECT CONCAT('ORD-', MAX(CAST(SUBSTRING(reference, 5) AS UNSIGNED)) + 1)
  INTO p_order_reference FROM medication_orders;

  -- trg_order_before_insert fills in total = subtotal + delivery_fee.
  INSERT INTO medication_orders (reference, subtotal, delivery_fee, delivery_address, patient_id,
prescription_id)
  VALUES (p_order_reference, v_subtotal, fn_delivery_fee(v_subtotal), p_address, v_patient_id,
v_prescription_id);
  COMMIT;
END$$
```

Ordering the new prescription from section 4

RX-2005 has Paracetamol × 12 (Rs 25) and Azithromycin × 3 (Rs 110), so the subtotal is 300 + 330 = 630 and the delivery fee is 40. Steam inhalation is not a catalogue medicine, so it is skipped.

```
SELECT name, price, stock FROM medicines WHERE id IN (1, 7);
CALL sp_place_order(@rx, '7 Park Street, Kolkata', @ord);
SELECT reference, status, subtotal, delivery_fee, total, patient_id, prescription_id FROM medication_orders
WHERE reference = @ord;
SELECT name, price, stock FROM medicines WHERE id IN (1, 7);
```

OUTPUT

name	price	stock
Paracetamol	25.00	490
Azithromycin	110.00	120

```
Query OK, 0 rows affected
```

reference	status	subtotal	delivery_fee	total	patient_id	prescription_id
ORD-3005	placed	630.00	40.00	670.00	2	5

name	price	stock
Paracetamol	25.00	478
Azithromycin	110.00	117

Rollback when the second medicine is short

RX-2003 needs Atorvastatin × 60 and then Aspirin × 60. Aspirin is set to 5 units. When the cursor reaches Aspirin it has already taken 60 Atorvastatin, but after the error Atorvastatin is back at 150 and the number of orders is unchanged.

```
UPDATE medicines SET stock = 5 WHERE name = 'Aspirin';
CALL sp_place_order('RX-2003', '12 MG Road, Bengaluru', @x);
SELECT name, stock FROM medicines WHERE name IN ('Atorvastatin', 'Aspirin');
SELECT COUNT(*) AS orders FROM medication_orders;
```

OUTPUT

```
Query OK, 1 row affected
ERROR 1644 (45000): Not enough stock of Aspirin (5 left, 60 needed)
```

name	stock
Atorvastatin	150
Aspirin	5

orders
5

Cursor 2: department report (sp_department_report)

The cursor goes through the departments one by one. For each, it counts the doctors and appointments and adds up the doctors' earnings with `fn_doctor_earnings`, collecting the rows in a temporary table that is returned at the end.

```
-- Department report. A CURSOR goes through the departments one by one and, for each,
-- counts its doctors and appointments and adds up its doctors' earnings (fn_doctor_earnings).
CREATE PROCEDURE sp_department_report()
BEGIN
  DECLARE v_id INT;
  DECLARE v_name VARCHAR(100);
  DECLARE v_active BOOLEAN;
  DECLARE v_done BOOLEAN DEFAULT FALSE;

  DECLARE dept_cursor CURSOR FOR SELECT id, name, is_active FROM departments ORDER BY name;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

  DROP TEMPORARY TABLE IF EXISTS tmp_department_report;
  CREATE TEMPORARY TABLE tmp_department_report (
    department   VARCHAR(100),
    active       VARCHAR(3),
    doctors      INT,
    appointments INT,
    earnings     DECIMAL(10,2)
  );

  OPEN dept_cursor;
  dept_loop: LOOP
    FETCH dept_cursor INTO v_id, v_name, v_active;
    IF v_done THEN
      LEAVE dept_loop;
    END IF;

    INSERT INTO tmp_department_report
    SELECT v_name, IF(v_active, 'Yes', 'No'),
           (SELECT COUNT(*) FROM doctors WHERE department_id = v_id),
           (SELECT COUNT(*) FROM appointments WHERE department_id = v_id),
           (SELECT COALESCE(SUM(fn_doctor_earnings(id)), 0) FROM doctors WHERE department_id = v_id);
  END LOOP;
  CLOSE dept_cursor;

  SELECT * FROM tmp_department_report;
  DROP TEMPORARY TABLE tmp_department_report;
END$$
```

Running the report

```
CALL sp_department_report();
```

OUTPUT

department	active	doctors	appointments	earnings
Cardiology	Yes	2	2	1980.00
Dermatology	Yes	1	2	0.00
General Medicine	Yes	2	3	500.00
Neurology	No	1	0	0.00
Orthopaedics	Yes	1	1	800.00
Paediatrics	Yes	1	1	600.00

Triggers

Trigger	Fires	Rule it enforces
`trg_appointment_before_insert`	BEFORE INSERT on appointments	The doctor must work in the chosen department, and the department must be active
`trg_appointment_before_update`	BEFORE UPDATE on appointments	Status can only change from `booked`; a finished visit can't be reopened
`trg_prescription_before_insert`	BEFORE INSERT on prescriptions	A new version must replace an active prescription of the same patient; sets `version` automatically
`trg_order_before_insert`	BEFORE INSERT on medication_orders	Computes `total`; the prescription must be active and belong to the same patient

These rules involve *other rows or tables* (the doctor's department, the old prescription), which a `CHECK` constraint cannot look at. That is why they are triggers. All four are `BEFORE` triggers so they can reject the row or correct `NEW` values before anything is written.

```
-- A new appointment must be in the doctor's own department, and that department must be open.
CREATE TRIGGER trg_appointment_before_insert
BEFORE INSERT ON appointments
FOR EACH ROW
BEGIN
  DECLARE v_doctor_department INT;
  DECLARE v_active BOOLEAN;

  SELECT department_id INTO v_doctor_department FROM doctors WHERE id = NEW.doctor_id;
  IF v_doctor_department <> NEW.department_id THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The doctor does not work in that department';
  END IF;

  SELECT is_active INTO v_active FROM departments WHERE id = NEW.department_id;
  IF NOT v_active THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'That department is not taking appointments';
  END IF;
END$$
```

Rejected: Dr. Ananya Rao (Cardiology) booked under General Medicine

```
INSERT INTO appointments (reference, starts_at, ends_at, patient_id, doctor_id, department_id)
VALUES ('APT-2000', '2026-11-20 10:00:00', '2026-11-20 10:20:00', 1, 1, 5);
```

OUTPUT

```
ERROR 1644 (45000): The doctor does not work in that department
```

Rejected: Neurology is not taking appointments

The procedure inserts the row, and the trigger stops it.

```
CALL sp_book_appointment(3, 8, '2026-11-12 10:00:00', 'Headache', @x);
```

OUTPUT

```
ERROR 1644 (45000): That department is not taking appointments
```

```
-- Appointment status can only move forward from 'booked'
-- (booked -> completed / cancelled / no_show); a finished appointment can't be changed back.
CREATE TRIGGER trg_appointment_before_update
BEFORE UPDATE ON appointments
FOR EACH ROW
BEGIN
  IF NEW.status <> OLD.status AND OLD.status <> 'booked' THEN
    SET @msg = CONCAT('Appointment ', OLD.reference, ' is already ', OLD.status, ' and cannot be changed to
', NEW.status);
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = @msg;
  END IF;
END$$
```

Allowed: booked → cancelled

```
UPDATE appointments SET status = 'cancelled' WHERE reference = 'APT-1008';
```

OUTPUT

```
Query OK, 1 row affected
```

Rejected: reopening a completed visit

```
UPDATE appointments SET status = 'booked' WHERE reference = 'APT-1001';
```

OUTPUT

```
ERROR 1644 (45000): Appointment APT-1001 is already completed and cannot be changed to booked
```

```
-- A new version of a prescription must replace an active prescription of the same patient,
-- and gets the next version number automatically.
CREATE TRIGGER trg_prescription_before_insert
BEFORE INSERT ON prescriptions
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);
  DECLARE v_version INT;
  DECLARE v_patient_id INT;

  IF NEW.supersedes_id IS NOT NULL THEN
    SELECT status, version, patient_id INTO v_status, v_version, v_patient_id
    FROM prescriptions WHERE id = NEW.supersedes_id;
    IF v_status <> 'active' THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only an active prescription can be amended';
    END IF;
    IF v_patient_id <> NEW.patient_id THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'The new version must be for the same patient';
    END IF;
    SET NEW.version = v_version + 1;
  END IF;
END$$
```

Rejected: amending a prescription that was already replaced

```
CALL sp_amend_prescription('RX-2001', @x);
```

OUTPUT

```
ERROR 1644 (45000): Only an active prescription can be amended
```

```
-- The order total is always computed as subtotal + delivery fee, and an order that cites a
-- prescription must use an active prescription of the same patient.
CREATE TRIGGER trg_order_before_insert
BEFORE INSERT ON medication_orders
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);
  DECLARE v_patient_id INT;

  SET NEW.total = NEW.subtotal + NEW.delivery_fee;

  IF NEW.prescription_id IS NOT NULL THEN
    SELECT status, patient_id INTO v_status, v_patient_id
    FROM prescriptions WHERE id = NEW.prescription_id;
    IF v_patient_id <> NEW.patient_id THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'That prescription belongs to another patient';
    END IF;
    IF v_status <> 'active' THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only an active prescription can be ordered';
    END IF;
  END IF;
END$$
```

The trigger corrects a wrong total

The INSERT says total = 1, but the trigger replaces it with 100 + 40.

```
INSERT INTO medication_orders (reference, subtotal, delivery_fee, total, delivery_address, patient_id)
VALUES ('ORD-3900', 100, 40, 1, '3 Sector 15, Noida', 4);
SELECT reference, subtotal, delivery_fee, total FROM medication_orders WHERE reference = 'ORD-3900';
```

OUTPUT

```
Query OK, 1 row affected
```

reference	subtotal	delivery_fee	total
ORD-3900	100.00	40.00	140.00

Rejected: ordering against another patient's prescription

```
INSERT INTO medication_orders (reference, subtotal, delivery_fee, total, delivery_address, patient_id,
prescription_id)
VALUES ('ORD-3901', 100, 40, 140, '12 MG Road, Bengaluru', 1, 2);
```

OUTPUT

```
ERROR 1644 (45000): That prescription belongs to another patient
```

All triggers in the database

```
SELECT TRIGGER_NAME, ACTION_TIMING, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = 'a10_hospital' ORDER BY EVENT_OBJECT_TABLE;
```

OUTPUT

TRIGGER_NAME	ACTION_TIMING	EVENT_MANIPULATION	EVENT_OBJECT_TABLE
trg_appointment_before_insert	BEFORE	INSERT	appointments
trg_appointment_before_update	BEFORE	UPDATE	appointments
trg_order_before_insert	BEFORE	INSERT	medication_orders
trg_prescription_before_insert	BEFORE	INSERT	prescriptions